Atelier 3 — Les bases de Git

Corrigé pratique | Débutant | Durée indicative : 1 h 30

Document : Corrigé rédigé à partir des supports fournis

Objectif

Créer un dépôt Git local, suivre les fichiers, préparer les modifications dans la zone de staging, créer plusieurs commits explicites et consulter l'historique. Les commandes ci-dessous sont à lancer dans un terminal.

Partie 1 — Configuration et création du projet

```
git --version
git config --global user.name "Votre Nom"
git config --global user.email "votre@email.com"
mkdir atelier-git
cd atelier-git
git init
git status
```

Remplacer le nom et l'adresse e-mail par ses propres informations. git init crée les métadonnées du dépôt dans le dossier courant ; git status indique l'état des fichiers et de la branche.

Réponses aux questions

À quoi sert git init ? À initialiser un dépôt Git local dans le dossier courant. **Rôle de git config :** enregistrer notamment le nom et l'adresse e-mail associés aux commits. **Working directory :** répertoire de travail contenant les fichiers tels qu'ils sont modifiés localement.

Partie 2 — Suivi des fichiers et premier commit

Créer un fichier index.html contenant par exemple :

```
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>Mon premier projet Git</title>
</head>
<body>
  <h1>Mon premier projet Git</h1>
</body>
</html>
```

```
git status
git add index.html
git status
git commit -m "Initialisation du projet"
git log
```

git add place le contenu choisi dans la zone de staging, c'est-à-dire la préparation du prochain commit. **git commit** enregistre un instantané des changements préparés dans l'historique local. L'option -m fournit le message du commit sans ouvrir d'éditeur interactif.

Partie 3 — Modifier un fichier

Ajouter un paragraphe dans index.html, par exemple : « *Cette application présente des annonces et permet de découvrir le fonctionnement de Git.* » Puis observer les changements avant et après staging.

```
git status
git diff
git add index.html
git diff --staged
git commit -m "Ajout de la présentation"
git log --oneline
```

Réponses aux questions

• **Que montre git diff ?** Les modifications non préparées entre le répertoire de travail et la zone de staging.

• **Différence entre git diff et git diff --staged :** le premier montre les changements non ajoutés au staging ; le second montre les changements déjà préparés par git add par rapport au dernier commit.

• **Pourquoi plusieurs commits ?** Pour séparer les changements par objectif, faciliter la relecture et retrouver ou annuler une étape plus facilement.

Partie 4 — Gérer plusieurs fichiers

Créer les deux fichiers suivants.

```
/* style.css */
body { font-family: Arial, sans-serif; margin: 2rem; }
h1 { color: #245a81; }

# README.md
# Atelier Git
Petit projet d’apprentissage : suivi des fichiers HTML et CSS avec Git.
```

```
git status
git add style.css README.md
git status
git commit -m "Ajout du style et de la documentation"
git log --oneline
```

Réponses aux questions

• **Tracked / untracked :** un fichier tracked est déjà connu de Git (commité ou préparé dans l’historique) ; un fichier untracked est présent dans le dossier mais pas encore suivi.

• **Ajouter plusieurs fichiers :** oui, par exemple avec git add style.css README.md, ou tous les changements pertinents avec git add . en vérifiant auparavant ce qui sera ajouté.

• **Rôle de README.md :** présenter le projet, ses objectifs, son installation et son utilisation.

Partie 5 — Bilan et vérification

Tableau récapitulatif des commandes

Commande	Rôle
git init	Initialiser un dépôt Git local.
git status	Consulter l’état des fichiers, du staging et de la branche.
git add	Préparer un ou plusieurs fichiers pour le prochain commit.
git commit -m "message"	Créer un commit avec un message explicite.
git log	Consulter l’historique détaillé des commits.
git log --oneline	Consulter l’historique dans un format compact, une ligne par commit.
git diff	Voir les changements non préparés.
git diff --staged	Voir les changements préparés pour le prochain commit.

Vérification finale

```
git status
ls
git log --oneline
```

Le dossier atelier-git doit contenir index.html, style.css et README.md. Le dépôt doit être initialisé et contenir au moins trois commits ayant les messages : « Initialisation du projet », « Ajout de la présentation » et « Ajout du style et de la documentation ». Une capture de la sortie de git log --oneline constitue le livrable demandé.

Astuce : avant chaque commit, exécuter git status et vérifier le diff. Cela réduit le risque d’inclure un fichier ou une modification qui n’était pas prévu.